```
  flipState(): void {
    this.visibleValue = !this.visibleValue
  }

  updateHiddenClass(): void {
    this.elementToHideTarget.classList.toggle("hidden", !this.visibleValue)
  }

  newText(): string {
    return this.visibleValue ? "Hide" : "Show"
  }

  updateText(): void {
    this.elementWithTextTarget.innerText = this.newText()
  }
}
```

The code has gotten a lot longer, in part because I've been aggressive about splitting off small methods, but it's only a small step more complex than the last version.

We start by declaring two targets: the button text and the element to hide. We also declare one Stimulus value, `visible`, and the associated TypeScript property `visibleValue`.

Our action method is `toggle`, which does three things: flip the state, update the hidden class, and update the text. We need to flip the state first because the target and text change depend on the new state.

The `flipState` method uses the value to set the visibility value of the button to its boolean opposite. The `updateHiddenClass` method uses the class list method `toggle` in its two-argument form, meaning that the class name will be added to the class list if the second argument, `this.visibleValue`, is true, and removed otherwise. The `updateText` method similarly sets the `innerText` of that DOM element to "Show" or "Hide" based on whether the toggle is active.

Finally, we're using the `connect` method to ensure that the initial state of the toggle is valid. The `connect` method adjusts the target and the text based on the initial state of the `active` data map key—since `connect` doesn't also call `flipState`, the state as